

```
<!--APRENDIZAJE AUTOMÁTICO-->
```

Microsoft Malware Prediction

}

```
<Por="Carmen Johana Calderón Chona  
Mateo Cañavera Aluma  
Miller Alexis Quintero García"/>
```



Contenido

- 01 Contexto del problema
- 02 Pregunta investigativa
- 03 Planteamiento
- 04 EDA
- 05 Limpieza y Transformación
- 06 Ingeniería de características
- 07 Ajuste de hiperparámetros
- 08 Entrenamiento de modelos
- 09 Evaluación y validación de métricas para el LightGBM
- 10 Conclusiones

Contexto del problema {

En el campo de la ciberseguridad, uno de los grandes desafíos es la detección temprana de dispositivos potencialmente vulnerables a infecciones por malware (software malicioso). Microsoft, como uno de los principales proveedores de sistemas operativos, recopila datos de telemetría de millones de dispositivos Windows a través de herramientas como Windows Defender. Esta información incluye características técnicas del dispositivo. Con estos datos, Microsoft ha creado un dataset que incluye registros de millones de dispositivos, algunos de los cuales han sido infectados por malware, mientras que otros permanecieron seguros.

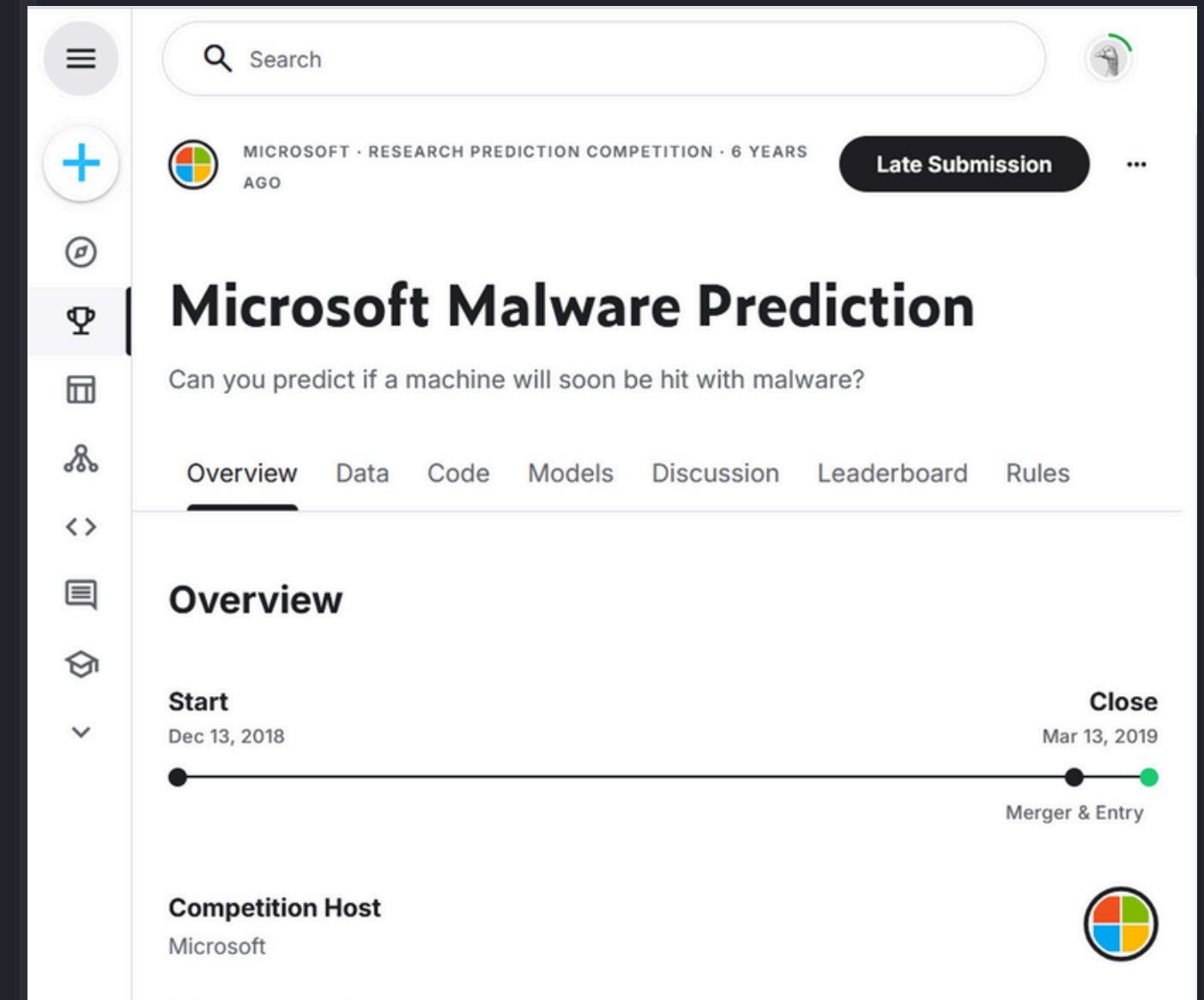


}

Pregunta investigativa

{

¿Es posible predecir de forma precisa la detección de amenazas en dispositivos Windows usando técnicas de aprendizaje automático y reducción de dimensionalidad con distintos enfoques?



Enlace al reto:
<https://www.kaggle.com/competitions/microsoft-malware-prediction>

}

Planteamiento y resolución del problema {

La metodología usada para abordar este problema incluyó el desarrollo de distintas etapas basadas en la ruta estándar para la creación y el entrenamiento de modelos de aprendizaje automático

Diagrama Gantt Proyecto:
Predictor de Malware basado en registros de Microsoft



Análisis exploratorio {

A partir del análisis previo de los datos se buscó estudiar el comportamiento de cada variable para establecer un esquema de preprocesamiento eficiente.

También se determinó cuáles serían los modelos a entrenar y de qué manera, teniendo en cuenta las características del problema

Total de columnas: 80 (sin contar el ID y el target)

Análisis exploratorio y preprocesamiento de los datos

Comenzaremos, por la exploración de los valores de las columnas:

La información mostrada aquí se basa en observaciones hechas al dataframe train.csv ejecutando el código `exploracionColumnas.py` y, parcialmente, en la información recolectada a través de las siguientes páginas web:

- <https://www.dtonomy.com/will-your-machine-be-hit-by-a-malware-soon/#ds>
- <https://www.kaggle.com/competitions/microsoft-malware-prediction/data>

Columnas del dataframe dado por train.csv

1. **'MachineIdentifier'**: Identificador único de la máquina

```
*** Análisis para columna: MachineIdentifier ***
MachineIdentifier
ffff75ba4f33d938ccfdb148b8ea16      1.0
000028988387b115f69f31a3bf04f09    1.0
00007535c3f730efa9ea0b7ef1bd645    1.0
00007905a28d863f6d0d597892cd692    1.0
0000b11598a75ea8ba1beea8459149f    1.0
...
000027c68b89acb49d4017763b043449    1.0
0000258d2b847c7549150cfec6464473    1.0
000024872c81cf03fa862aa8f99e0984    1.0
00001f26e9e5775277d6231fc6ac9e70    1.0
00001b924fcc6922321cfadbafd8a91a    1.0
Length: 8921483, dtype: float64
La cantidad de valores nulos es 0
El tipo de datos es -> object
```

2. **ProductName**: Nombre del producto antivirus de Microsoft instalado (ej: 'mse', 'wdav', etc.). Usualmente: 'Windows Defender Antivirus'.

49

Categorías

13

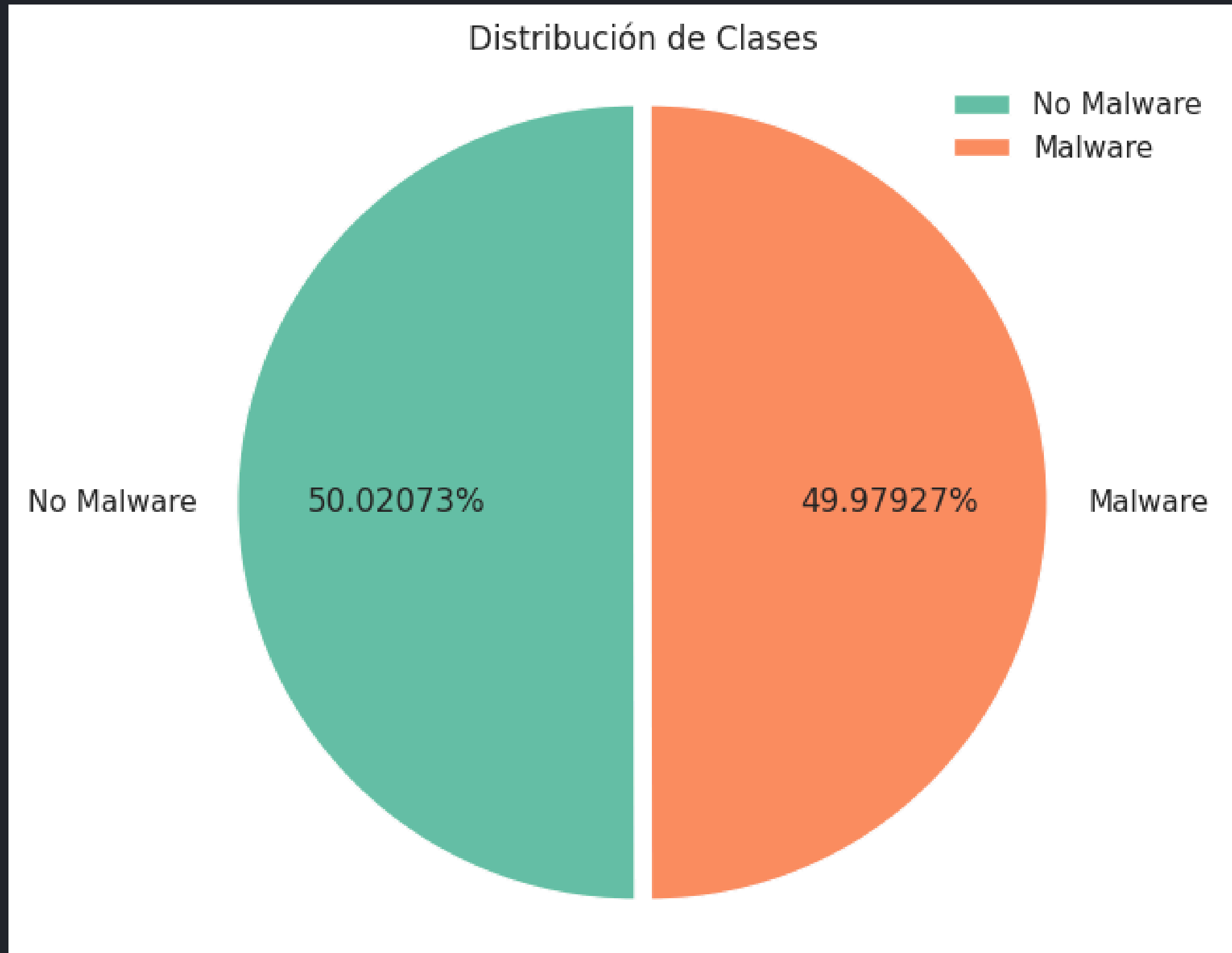
Numéricas

18

Booleanas

}

Distribución de HasDetection {



}

Limpieza y transformación de datos {

Esta etapa del proyecto consistió en :

- Eliminación columnas irrelevantes o con información única por fila, como MachineIdentifier.
- Reducción de la cantidad de valores únicos en columnas con muchas *clases* con la intención de aumentar la eficiencia del preprocesamiento.
- Investigación a profundidad de cada una de las columnas y su relación con la variable objetivo HasDetection.
- Imputación de valores nulos en algunas de las variables categóricas de texto mediante la técnica de Target Encoding y de valores nulos en variables numéricas con la media añadiendo una variable indicadora

```
In [35]: # Visualización de columnas con datos faltantes:
print("Columnas con valores faltantes:")
print(null_values[null_values > 0])
```


Columnas con valores faltantes:	
RtpStateBitfield	32318
DefaultBrowsersIdentifier	8488045
AVProductStatesIdentifier	36221
AVProductsInstalled	36221
AVProductsEnabled	36221
CityIdentifier	325409
OrganizationIdentifier	2751518
GeoNameIdentifier	213
OsBuildLab	21
IsProtected	36044
PuaMode	8919174
SMode	537759
IeVerIdentifier	58894
SmartScreen	3177011
Firewall	91350
UacLuaenable	10838
Census_OEMNameIdentifier	95478
Census_OEMModelIdentifier	102233
Census_ProcessorCoreCount	41306
Census_ProcessorManufacturerIdentifier	41313
Census_ProcessorModelIdentifier	41343
Census_ProcessorClass	8884852
Census_PrimaryDiskTotalCapacity	53016
Census_PrimaryDiskTypeName	12844
Census_SystemVolumeTotalCapacity	53002
Census_TotalPhysicalRAM	80533
Census_ChassisTypeName	623
Census_InternalPrimaryDiagonalDisplaySizeInInches	47134
Census_InternalPrimaryDisplayResolutionHorizontal	46986
Census_InternalPrimaryDisplayResolutionVertical	46986

59,354,742

total de nulos

}

}

Ingeniería de características {

Se hizo el análisis de variables categóricas de alta cardinalidad para reducir dimensiones (por ejemplo, agrupando por frecuencia o seleccionando las categorías más comunes o generales)

Además, se incorporaron columnas binarias adicionales que marcan la ausencia de datos originales, permitiendo al modelo capturar información asociada a valores faltantes.

```
# 4. Preprocesamiento: combinación de OneHot y TargetEncoder
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False), onehot_cols),
        #('target', TargetEncoder(cols=target_cols), target_cols)
        ('target', TargetEncoder(smooth="auto"), target_cols)
    ],
    remainder='passthrough'
)
```

}

Entrenamiento de modelos {

Se entrenaron tres modelos tras realizar el preprocesamiento: LightGBM, RandomForest y Logistic Regression



LightGBM (baseline seleccionado)

- AUC: 0.7079
- Accuracy: 0.6472
- F1: 0.6404

Random Forest

- AUC: 0.7009
- Accuracy: 0.6410

Logistic Regression

- AUC: 0.5456
- Accuracy: 0.5299

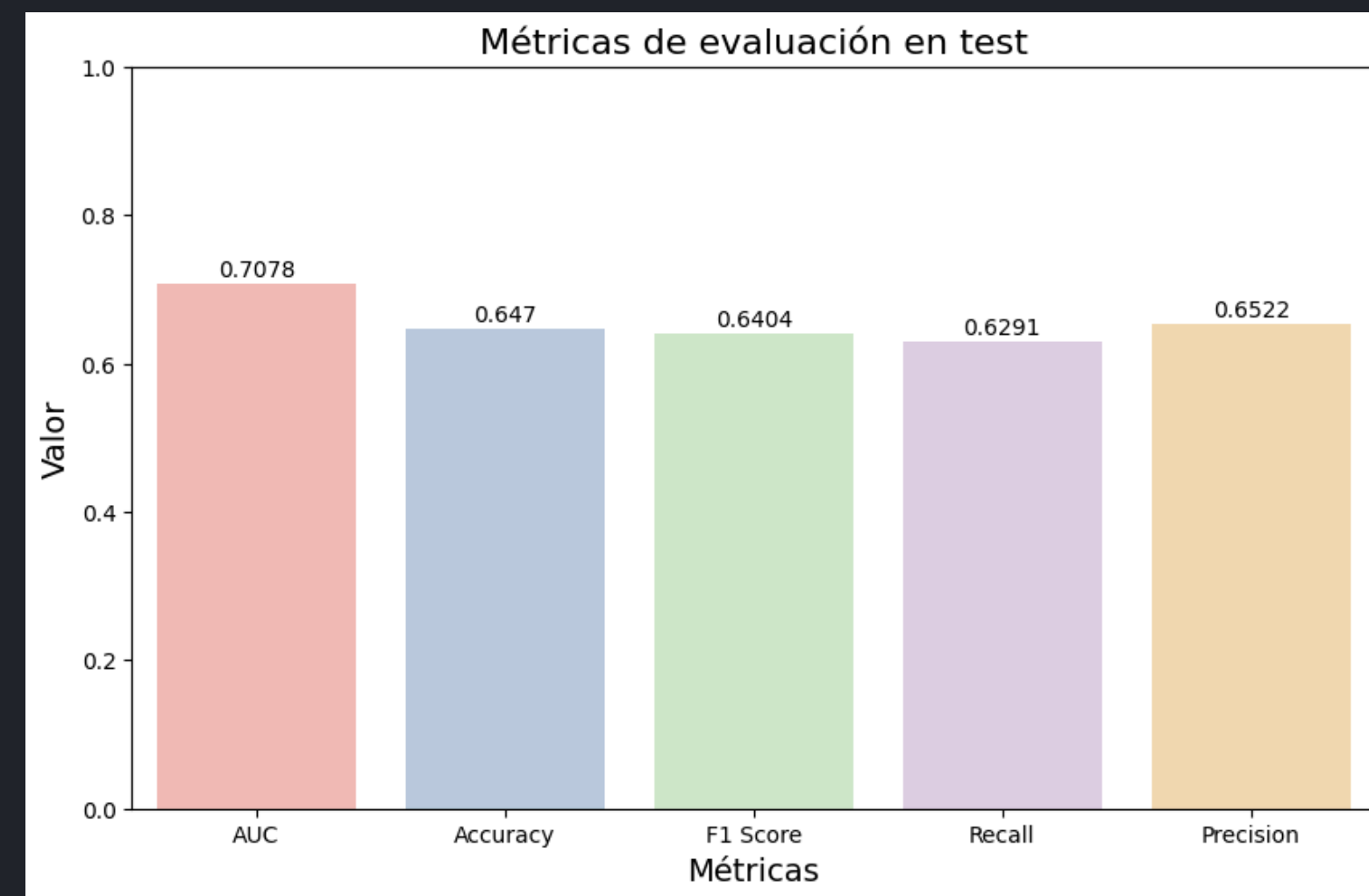
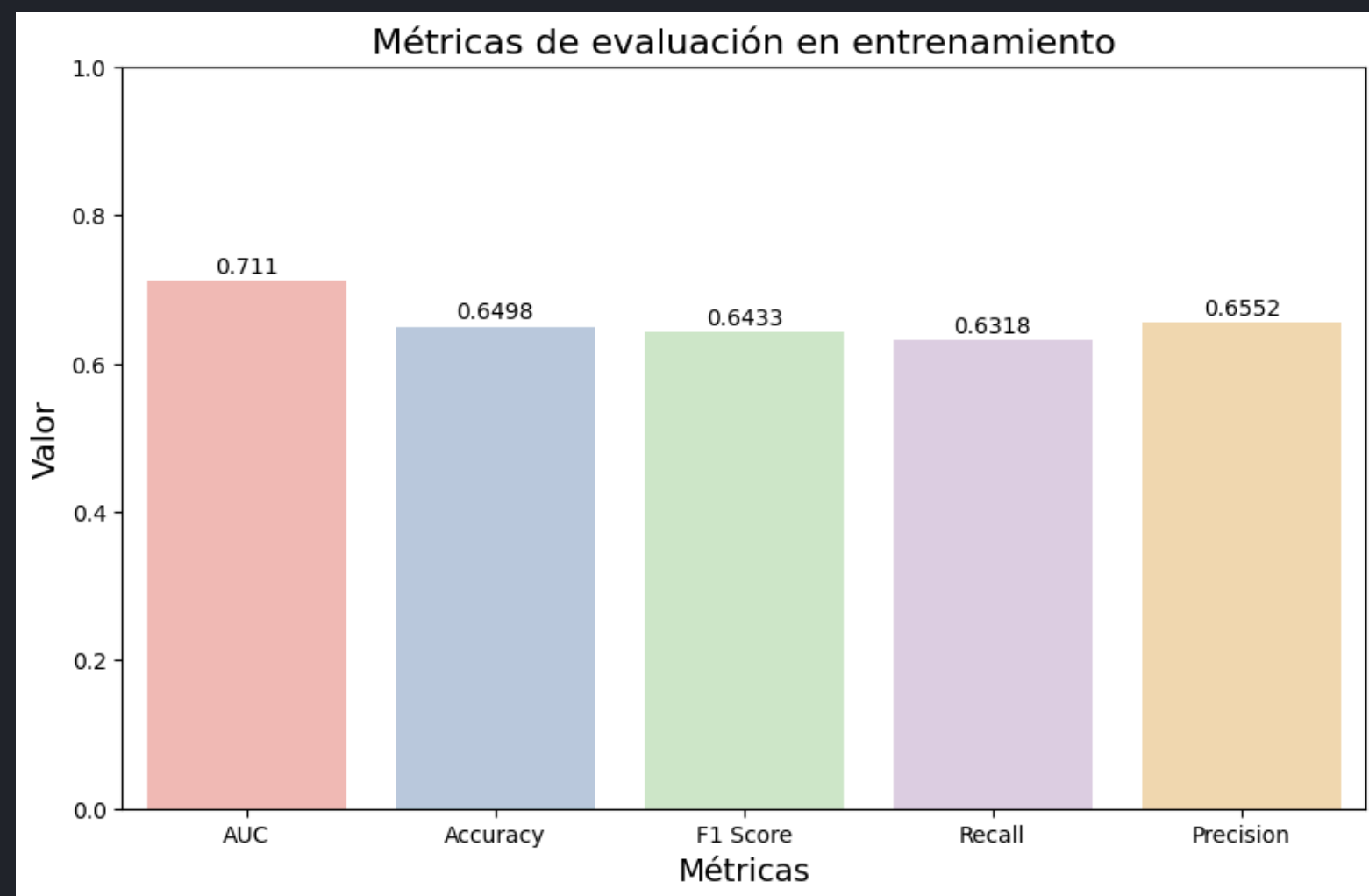
}

Ajuste de hiperparámetros {

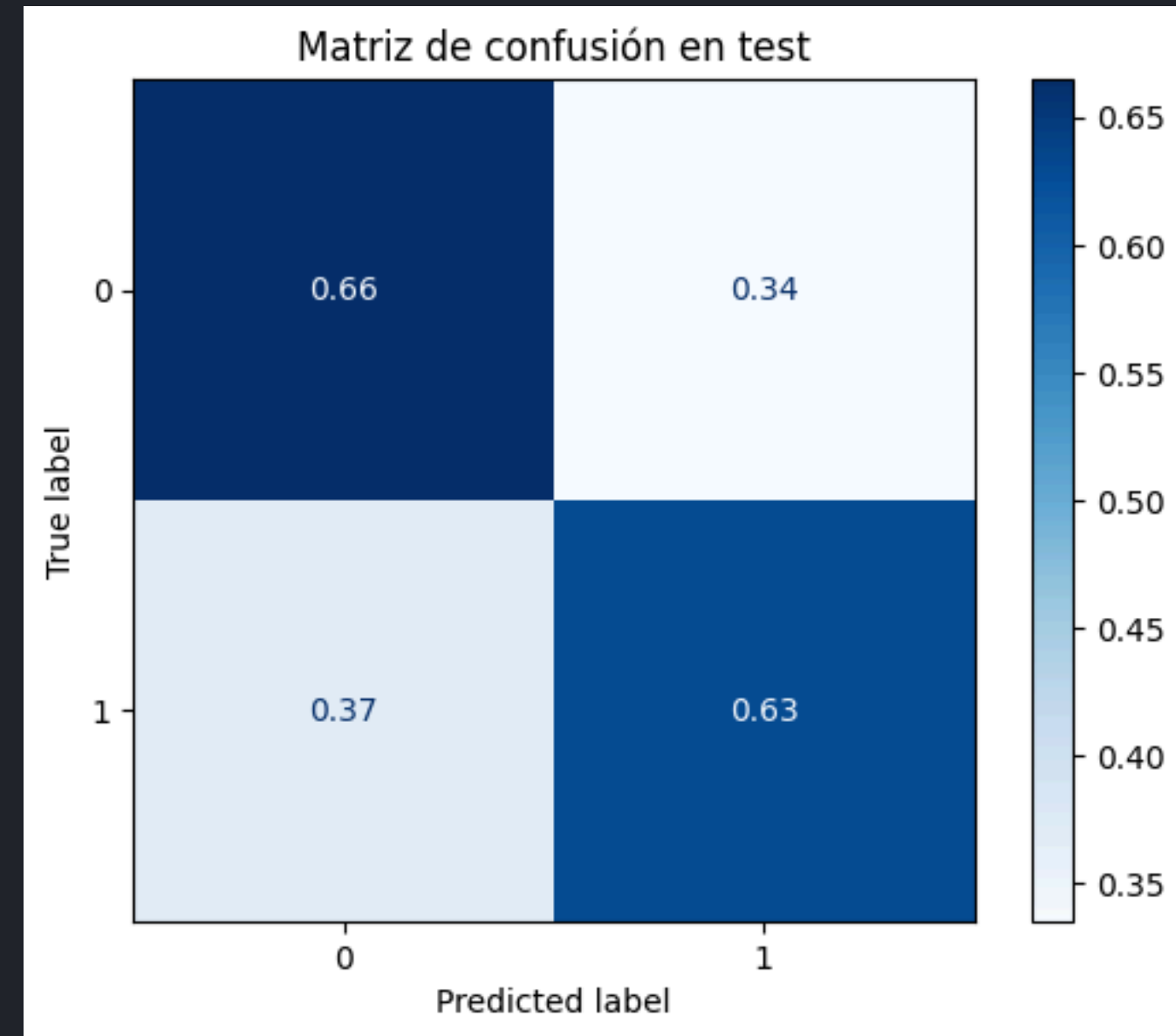
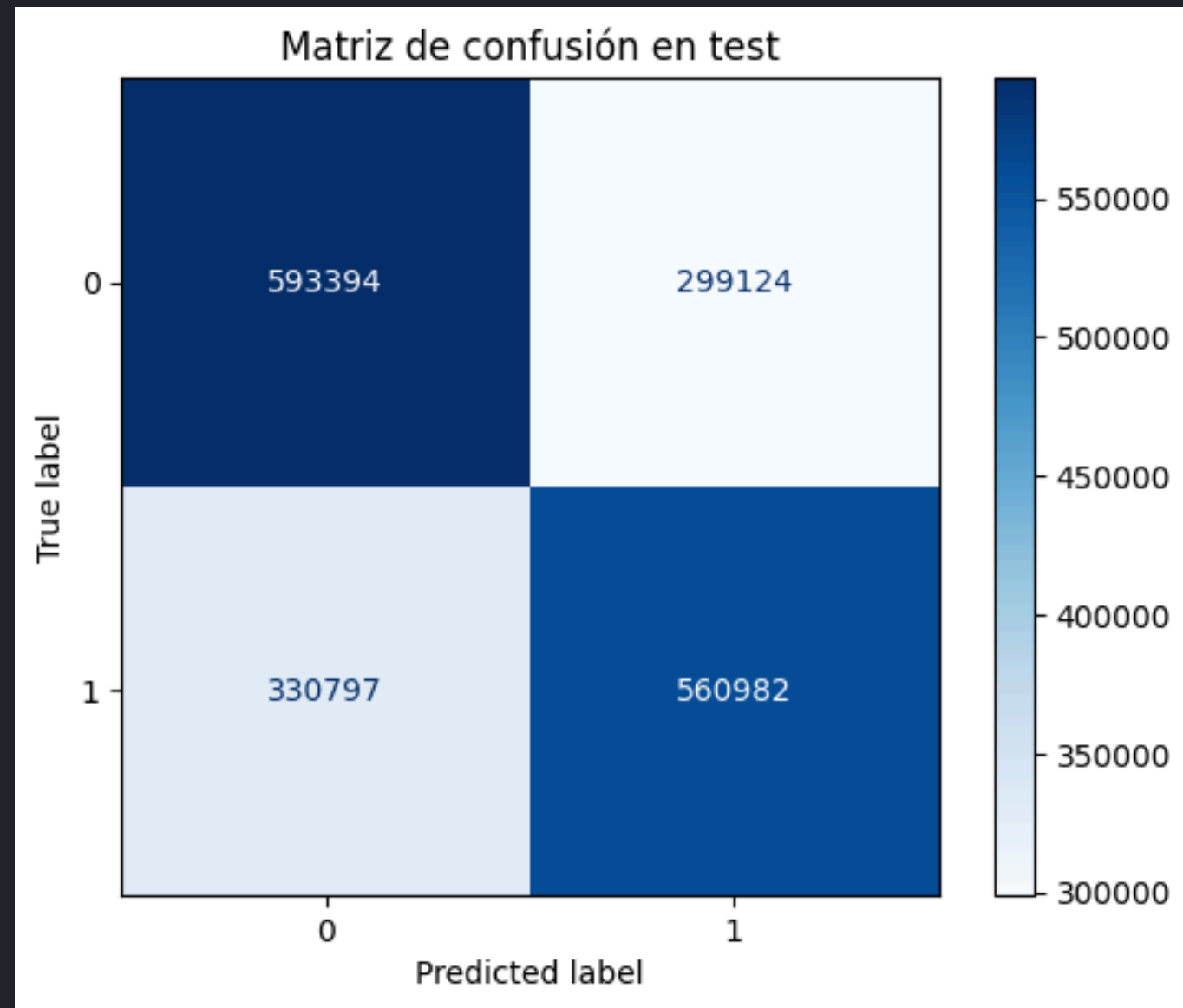
Se hizo el experimento definiendo una función objetivo a partir de la media de tres folds de validación cruzada y tres trials de Optuna (9 iteraciones), pero los recursos a disposición imposibilitaron su completa ejecución

ERROR : (}

Evaluación y validación con métricas del LightGBM {



Matriz de Confusión {



}

Conclusiones {

- Es posible predecir con buena precisión la probabilidad de que un dispositivo esté infectado por malware utilizando modelos de aprendizaje automático aplicados sobre datos reales recolectados por Microsoft
- La limpieza y transformación de datos fue clave para manejar inconsistencias, valores faltantes y categorías de alta cardinalidad que podrían afectar el rendimiento del modelo.
- El LightGBM baseline fue el modelo de mejor rendimiento general y se toma como referencia principal por su accuracy
- Si bien el modelo puede ser útil como herramienta de apoyo, no reemplaza sistemas de protección en tiempo real, y su efectividad depende de la calidad y actualidad de los datos

}

```
<!--APRENDIZAJE AUTOMÁTICO-->
```

Gracias {

}